

OS MINI PROJECT REPORT

MOHAMMED IBRAHIM

IMT2023112

Admin Id: admin@iiitb
Admin Password: AdmiN.in

Problem Statement

The "Academia" project has the goal of creating a multi-purpose, easy-to-use course registration system. The system will enable the exchange of information between students, instructors, and administrators, with an emphasis on secure access and streamlined management of course data. The system will meet the needs of:

- **Data Storage:** Persistent storage of student, faculty, and course information in files.
- **User Authentication:** A login system to authenticate users (students, faculty, and administrators) before granting access to their respective accounts.
- **Role-Based Access Control:** Implementation of distinct roles (student, faculty, admin) with specific functionalities and permissions.
- **Secure Administration:** Password-protected administrative access to prevent unauthorized modification of the system.
- **Course Management:** Functionalities for adding, removing, and viewing course details, including enrollment management.
- **Student Management:** Functionalities for students to enroll in and unenroll from courses, view their enrolled courses, and manage their passwords.
- **Faculty Management:** Functionalities for faculty to add/remove courses and view course enrollments.
- **Concurrency Control:** Implementation of read/write locks to protect critical data sections during concurrent access, specifically for course enrollment operations.
- **Client-Server Architecture:** A client-server architecture using socket programming, where the server manages the database and serves multiple concurrent client requests.
- **Operating System Integration:** Utilization of system calls for process management, file management, file locking, semaphores, multithreading, and inter-process communication.

Implementation Details

The project's implementation encompasses the following key elements:

- **File-Based Data Storage:** The system stores student, faculty, and course data in files. This will likely involve creating and managing data files on the server.
- **User Authentication:**
 - The server handles user authentication.
 - Clients (student, faculty, admin) send login credentials to the server.
 - The server verifies the credentials against stored user data.
 - Upon successful login, the server provides the client with access to their role-specific functionalities.
- **Role-Based Functionalities:**
 - **Administrator:**
 - Add/update student and faculty details.
 - Activate/deactivate student accounts.
 - The Admin.c and Admin.h files implement these features.
 - **Student:**
 - Enroll in/unenroll from courses.
 - View enrolled courses.
 - Change password.
 - The Student.c and Student.h files implement these features.
 - **Faculty:**
 - Add/remove courses.
 - View course enrollments.
 - Change password.
 - The Faculty.c and Faculty.h files implement these features.
- **Concurrency Control:**
 - The server uses semaphores and file locking to manage concurrent access to course data.
 - Read locks are applied when viewing course details.
 - Write locks are used during enrollment and unenrollment to prevent data corruption.
- **Client-Server Communication:**
 - The Client.c and Client.h files implement the client-side logic for communicating with the server.
 - The Server.c and Server.h files implement the server-side logic for handling client connections and requests.
 - Socket programming is used for communication between clients and the server.

- **System Calls:** The project emphasizes using system calls for OS functionalities. This means file operations, process management, and inter-process communication are implemented using low-level system calls (e.g., open, read, write, socket, bind, listen, accept, fork, pthread_create, sem_wait, sem_post, and flock).

Source Code with Brief Explanation

Admin.h:

```
Admin.h

#ifndef ADMIN_H
#define ADMIN_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/file.h>
#include <semaphore.h>
#include <ctype.h>
#include "Utilities.h"

extern sem_t student_sem;
extern sem_t faculty_sem;

void add_student(int client_socket);
void view_student_details(int client_socket);
void add_faculty(int client_socket);
void view_faculty_details(int client_socket);
void activate_student(int client_socket);
void admin_menu(int client_socket);
void block_student(int client_socket);
void modify_student_details(int client_socket);
void modify_faculty_details(int client_socket);
void log_out(int client_socket);

#endif
```

- **Purpose:** Defines the interface for administrator-related functionalities.
- **Contents:**
 - Includes necessary header files for standard I/O, memory allocation, string manipulation, file operations, file locking, semaphores, and character type

checking.

- Declares external semaphore variables (student_sem, faculty_sem) used for synchronization.
- Declares function prototypes for administrator operations, such as adding/viewing student/faculty, activating/blocking students, and displaying the admin menu.
- add_student(int client_socket): Adds a new student to the system.
- view_student_details(int client_socket): Retrieves and displays details of one or more students.
- add_faculty(int client_socket): Adds a new faculty member to the system.
- view_faculty_details(int client_socket): Retrieves and displays details of one or more faculty members.
- activate_student(int client_socket): Activates a student account.
- admin_menu(int client_socket): Displays the main menu for the administrator.
- block_student(int client_socket): Blocks a student account.
- modify_student_details(int client_socket): Updates existing student information.
- modify_faculty_details(int client_socket): Updates existing faculty information.
- log_out(int client_socket): Handles the logout process for an administrator.

Client.h:

Client.h

```
#ifndef CLIENT_H
#define CLIENT_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <signal.h>
#include "Utilities.h"

#define PORT 8080
#define BUFFER_SIZE 1024
int client_socket;

void handle_sigint();

#endif
```

- **Purpose:** Defines the client-side interface and constants for communicating with the server.
- **Contents:**
 - Includes standard headers and the Utilities.h header.
 - Defines the server port and buffer size.
 - Declares the global client_socket variable.
 - Declares the handle_sigint function, which is used to handle the SIGINT signal (Ctrl+C) for graceful termination.

Faculty.h:

```
Faculty.h

#ifndef FACULTY_H
#define FACULTY_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/file.h>
#include "Utilities.h"

// Function prototypes for faculty functions
void faculty_menu(int client_socket, int faculty_id);
void view_offering_courses(int client_socket, int faculty_id);
void add_course(int client_socket, int faculty_id);
void remove_course(int client_socket, int faculty_id);
void update_course_details(int client_socket, int faculty_id);
```

- **Purpose:** Defines the interface for faculty-related functionalities.
- **Contents:**
 - Includes necessary header files.
 - Declares function prototypes for faculty operations, such as viewing offered courses, adding/removing courses, and displaying the faculty menu.
- `faculty_menu(int client_socket, int faculty_id)`: Displays the main menu for a faculty member.
- `view_offering_courses(int client_socket, int faculty_id)`: Retrieves and displays the courses offered by a specific faculty member.
- `add_course(int client_socket, int faculty_id)`: Adds a new course to the system.
- `remove_course(int client_socket, int faculty_id)`: Removes a course from the system.
- `update_course_details(int client_socket, int faculty_id)`: Updates details of a

course.

Server.h:

```
Server.h

#ifndef SERVER_H
#define SERVER_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <pthread.h>
#include <signal.h>
#include <semaphore.h>
#include "Server.h"
#include <asm-generic/socket.h>

#define PORT 8080
#define MAX_CLIENTS 10

extern sem_t student_sem;
extern sem_t faculty_sem;
extern sem_t course_sem;

void client_handler(void *arg);
void signal_handler(int sig);

#endif
```

- **Purpose:** Defines the server-side interface, constants, and external variables.
- **Contents:**
 - Includes necessary header files.
 - Defines the server port and the maximum number of clients.
 - Declares external semaphore variables (student_sem, faculty_sem, course_sem) for synchronizing access to shared data.
 - Declares function prototypes for the client_handler (which handles communication with individual clients) and signal_handler (which handles signals, such as program termination).
- client_handler(void *arg): Handles communication with a connected client in a separate thread.
- signal_handler(int sig): Handles system signals, such as program termination.

Student.h:

```
Student.h

#ifndef STUDENT_H
#define STUDENT_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/file.h>
#include "Utilities.h"

// Function prototypes for student functions
void student_menu(int client_socket, int student_id);
void view_all_courses(int client_socket);
void enroll_course(int client_socket, int student_id);
void drop_course(int client_socket, int student_id);
void view_enrolled_courses(int client_socket, int student_id);
```

- **Purpose:** Defines the interface for student-related functionalities.

- **Contents:**

- Includes necessary header files.
- Declares function prototypes for student operations, such as viewing all courses, enrolling in courses, dropping courses, and displaying the student menu.
- `student_menu(int client_socket, int student_id)`: Displays the main menu for a student.
- `view_all_courses(int client_socket)`: Retrieves and displays all available courses.
- `enroll_course(int client_socket, int student_id)`: Enrolls a student in a course.
- `drop_course(int client_socket, int student_id)`: Removes a student from a course.
- `view_enrolled_courses(int client_socket, int student_id)`: Retrieves and displays the courses in which a student is enrolled.

Utilities.h:

Purpose: Defines utility functions and constants used throughout the application.

Contents:

Includes the necessary header file `<semaphore.h>` for semaphore operations.

Declares structure definitions for:

- **Student:** Contains student ID (`id`), name (`name`), password (`password`), and an active status flag (`active`).
- **Faculty:** Contains faculty ID (`id`), name (`name`), and password (`password`).
- **Course:** Contains course ID (`id`), name (`name`), the ID of the faculty teaching it (`faculty_id`), the total number of seats (`total_seats`), the number of available seats (`available_seats`), an array to store the IDs of enrolled students (`enrolled_students`), and the current enrollment count (`enrollment_count`).

Declares external semaphore variables:

- `student_sem`: A semaphore for managing access to student-related resources.
- `faculty_sem`: A semaphore for managing access to faculty-related resources.
- `course_sem`: A semaphore for managing access to course-related resources.

Declares function prototypes for various utility functions:

- `authenticate(int client_socket, int role)`: Handles user authentication (student or faculty).
- `change_password(int client_socket, int id, int role)`: Allows users to change their

password.

- `exit_portal(int client_socket)`: Handles the complete exit of a client from the portal.
- `log_out(int client_socket)`: Handles the logout of a user, returning them to the login menu.
- `send_message_and_prompt(int socket, const char *message)`: Sends a message to the client and likely prompts for further input.
- `send_message(int socket, const char message)`: Sends a character message to the client.
- `receive_message(int socket, char buffer)`: Receives a character message from the client.

Output Screenshots

1. Client-Server connection

```
o mohammed-ibrahim@mi-on-ubuntu:~/Downloads/Work/Sem 4/OS/LDraft/DRAFT2$ ./server
rver
Initiating server...
Server available for connections...
Client Request: 127.0.0.1:50578
Client Connected
█

o mohammed-ibrahim@mi-on-ubuntu:~/Downloads/Work/Sem 4/OS/LDraft/DRAFT2$ ./client
ient
Requesting connection with server...
Request for connection successful.
Connected to server.

=== Academia Course Registration ===
Select Login Type:
1. Admin 2. Faculty 3. Student 4. Exit
Your Choice: █
```

2. Add student

```
=== Admin Menu ===
1. Add Student
2. View Student Details
3. Add Faculty
4. View Faculty Details
5. Activate Student
6. Block Student
7. Edit Student Details
8. Edit Faculty Details
9. Logout & Exit
Your Choice: 1
Enter student name: ibrahim
Enter student password: ibrahim.mi
Student with ID: 1
added.=== Admin Menu ===
1. Add Student
```

3. Edit student details

```
=== Admin Menu ===
1. Add Student
2. View Student Details
3. Add Faculty
4. View Faculty Details
5. Activate Student
6. Block Student
7. Edit Student Details
8. Edit Faculty Details
9. Logout & Exit
Your Choice: 7
Enter Student ID: 1
Enter new name: ibrahimnew
Enter new password: ibrahim.mi8
Student details updated.
=== Admin Menu ===
1. Add Student
```

4. Faculty

```
=== Faculty Menu ===
1. View Offered Courses
2. Add New Course
3. Remove Course from Catalog
4. Update Course Details
5. Change Password
6. Logout & Exit

Your Choice: █
```

5. Add course and change password

```
=== Faculty Menu ===
1. View Offered Courses
2. Add New Course
3. Remove Course from Catalog
4. Update Course Details
5. Change Password
6. Logout & Exit

Your Choice: 2
Enter course name: dsa
Enter total seats: 40
Course with ID: 1 added.

=== Faculty Menu ===
1. View Offered Courses
2. Add New Course
3. Remove Course from Catalog
4. Update Course Details
5. Change Password
6. Logout & Exit

Your Choice: 5
Enter current password: teacher.nice
Enter new password: teacher.notnice
Password changed successfully.

=== Faculty Menu ===
```

6. Student login

```
Logging out...  
=== Academia Course Registration ===  
Select Login Type:  
1. Admin 2. Faculty 3. Student 4. Exit  
Your Choice: 3  
Enter username: ibrahimnew  
Enter password: ibrahim.mi8
```

7. Course enrolling

```
Your Choice: 1
Courses:
ID: 1 | Name: dsa | Seats: 40

=== Student Menu ===
1. View Available Courses
2. Enroll in a Course
3. Drop a Course
4. View Enrolled Courses
5. Change Password
6. Logout & Exit

Your Choice: 2
Courses:
ID: 1 | Name: dsa | Seats: 40
Enter course ID: 1
Enrolled in the course with ID: 1

=== Student Menu ===
1. View Available Courses
2. Enroll in a Course
3. Drop a Course
4. View Enrolled Courses
5. Change Password
6. Logout & Exit

Your Choice: █
```

8. View and drop course

```
=== Student Menu ===
1. View Available Courses
2. Enroll in a Course
3. Drop a Course
4. View Enrolled Courses
5. Change Password
6. Logout & Exit

Your Choice: 4
Courses Enrolled:
ID: 1 | Name: dsa | Faculty ID: 1

=== Student Menu ===
1. View Available Courses
2. Enroll in a Course
3. Drop a Course
4. View Enrolled Courses
5. Change Password
6. Logout & Exit

Your Choice: 3
Courses enrolled:
ID: 1 | Name: dsa
Enter course ID: 1
Dropped course with ID: 1

=== Student Menu ===
1. View Available Courses
2. Enroll in a Course
3. Drop a Course
4. View Enrolled Courses
5. Change Password
6. Logout & Exit

Your Choice: █
```

9. Log out and Exit

=== Student Menu ===

1. View Available Courses
2. Enroll in a Course
3. Drop a Course
4. View Enrolled Courses
5. Change Password
6. Logout & Exit

Your Choice: 6

Logging out...

=== Academia Course Registration ===

Select Login Type:

1. Admin
2. Faculty
3. Student
4. Exit

Your Choice: 4

Exiting Portal...

mohammed-ibrahim@mi-on-ubuntu:~/Downloads/Work/S